# C++ Module 03

Traduction française — Inheritance (Héritage)

Sujet traduit avec conservation des termes techniques C++ et une mise en page inspirée du document original.

## Sommaire


- Chapitre I — Introduction
- Chapitre II — Règles générales
- Chapitre III — Instructions concernant l’IA
- Exercice 00 — Aaaaand... OPEN!
- Exercice 01 — Serena, my love!
- Exercice 02 — Repetitive work
- Exercice 03 — Now it’s weird!
- Chapitre VIII — Soumission et Peer-Evaluation


## Chapitre I — Introduction

C++ est un langage de programmation généraliste créé par Bjarne Stroustrup comme une extension du langage de programmation C, également appelé « C with Classes ».

L’objectif de ces modules est de te faire découvrir la programmation orientée objet (OOP — Object-Oriented Programming).

Ce sera le point de départ de ton apprentissage du C++. De nombreux langages permettent d’apprendre la programmation orientée objet, mais nous avons choisi C++ parce qu’il est dérivé de ton vieil ami C.

Comme ce langage est complexe, et afin de garder les choses simples, ton code devra respecter la norme C++98.

Nous savons que le C++ moderne est très différent sur de nombreux aspects. Si tu souhaites devenir une développeuse C++ compétente, il te faudra approfondir tes connaissances après le tronc commun de 42.

## Chapitre II — Règles générales

### Compilation

- Compile ton code avec c++ et les flags -Wall -Wextra -Werror.
- Ton code doit toujours compiler si tu ajoutes le flag -std=c++98.

### Formatage et conventions de nommage

- Les répertoires des exercices doivent être nommés ainsi : ex00, ex01, etc.
- Nomme tes fichiers, classes, fonctions, member functions et attributs comme indiqué dans les consignes.
- Les noms de classes doivent utiliser UpperCamelCase. Les fichiers contenant le code d’une classe doivent porter le nom de cette classe : ClassName.hpp, ClassName.h, ClassName.cpp ou ClassName.tpp. Par exemple, la classe BrickWall sera définie dans BrickWall.hpp.
- Sauf indication contraire, chaque message affiché doit se terminer par un caractère de nouvelle ligne (\n) et être envoyé sur la sortie standard (standard output).
- Aucun style de codage particulier n’est imposé dans les modules C++. Tu peux suivre celui que tu préfères, mais écris un code propre et lisible.

### Éléments autorisés et interdits

- Tu peux utiliser presque tout ce qui appartient à la standard library. Il est conseillé d'utiliser les équivalents C++ des fonctions C que tu connais.
- Les bibliothèques externes sont interdites. C++11 et ses versions dérivées, ainsi que Boost, sont interdits.
- Les fonctions *printf(), *alloc() et free() sont interdites. Leur utilisation entraîne une note de 0.
- Sauf indication contraire, using namespace et friend sont interdits. Leur utilisation entraîne une pénalité de -42.
- La STL (Standard Template Library) est autorisée uniquement dans les Modules 08 et 09. Avant cela, les Containers (vector, list, map, etc.) et les Algorithms nécessitant le header sont interdits. Leur utilisation entraîne une pénalité de -42.

### Exigences de conception

- Les memory leaks existent aussi en C++. Lorsque tu alloues de la mémoire avec new, tu dois éviter les fuites mémoire.
- Du Module 02 au Module 09, les classes doivent respecter l'Orthodox Canonical Form (OCF), sauf indication contraire.
- Toute implémentation de fonction placée dans un header file, sauf pour les function templates, entraîne une note de 0.
- Chaque header doit pouvoir être utilisé indépendamment et inclure toutes ses dépendances. Utilise des include guards pour éviter les inclusions multiples ; leur absence entraîne une note de 0.

### À lire attentivement

- Tu peux ajouter des fichiers supplémentaires pour mieux séparer ton code, à condition de rendre tous les fichiers obligatoires.
- Les exemples peuvent révéler des exigences qui ne sont pas explicitement écrites dans les consignes.
- Lis entièrement chaque module avant de commencer.
- Par Odin, par Thor ! Utilise ton cerveau !

Concernant le Makefile, les mêmes règles que pour les projets C s'appliquent aux projets C++.

Tu devras implémenter beaucoup de classes. Cela peut sembler fastidieux, sauf si tu sais automatiser certaines tâches avec ton éditeur de texte préféré.

Tu disposes d'une certaine liberté pour réaliser les exercices. Respecte les règles obligatoires et n'hésite pas à lire les concepts théoriques.

# Chapitre III — Instructions concernant l'IA

### Contexte

Ce projet est conçu pour t'aider à découvrir les bases fondamentales de ta formation à 42.

Pour acquérir les connaissances et compétences essentielles, utilise les outils d'IA et les aides disponibles de manière réfléchie.

Un apprentissage solide demande un véritable effort intellectuel : relever des défis, répéter les exercices et échanger avec les autres étudiants.

Pour en savoir plus sur la position de 42 concernant l'IA — comme outil d'apprentissage, dans la formation et dans le monde professionnel — consulte la FAQ dédiée sur l'intranet.

### Message principal

- Construire des bases solides sans prendre de raccourcis.
- Développer réellement ses compétences techniques et ses capacités de réflexion.

- Pratiquer le peer-learning, apprendre à apprendre et à résoudre de nouveaux problèmes.
- Comprendre que le parcours d'apprentissage est plus important que le résultat.
- Apprendre les risques liés à l'IA et développer des méthodes de contrôle efficaces pour éviter les erreurs courantes.

## Règles pour les apprenants

- Réfléchis aux tâches qui te sont confiées, particulièrement avant de faire appel à l'IA.
- Ne demande pas directement les réponses à l'IA.
- Renseigne-toi sur l'approche globale de 42 concernant l'IA.

## Résultats attendus

- Acquérir de bonnes bases techniques et en programmation.
- Comprendre pourquoi et comment l'IA peut être dangereuse pendant cette phase.

## Commentaires et exemples

Oui, nous savons que l'IA existe et qu'elle peut résoudre tes projets. Mais tu es ici pour apprendre, pas pour démontrer que l'IA a appris. Ne perds pas ton temps — ni le nôtre — à prouver que l'IA peut résoudre le problème donné.

À 42, apprendre ne consiste pas à connaître la réponse, mais à développer la capacité à la trouver. L'IA fournit directement la réponse, mais cela t'empêche de construire ton propre raisonnement. Raisonner demande du temps, des efforts et implique parfois des échecs.

Pendant les examens, l'IA n'est pas disponible : pas d'Internet, pas de smartphone, etc. Tu réaliseras rapidement si tu as trop compté sur l'IA pendant ton apprentissage.

Le peer-learning fait découvrir différentes idées et approches, améliore les compétences relationnelles et la capacité à envisager plusieurs solutions. N'hésite pas à parler, à poser des questions et à apprendre avec les autres.

L'IA fera partie du programme, comme outil d'apprentissage et comme sujet d'étude. Tu auras même l'occasion de créer ton propre logiciel d'IA. Consulte la documentation sur l'intranet.

## Bonne pratique

« Je suis bloquée sur un nouveau concept. Je demande à quelqu'un comment il s'y est pris. Nous discutons pendant 10 minutes et, soudain, je comprends. »

## Mauvaise pratique

« J'utilise secrètement l'IA et je copie du code qui semble correct. Pendant la peer-evaluation, je suis incapable d'expliquer quoi que ce soit. J'échoue. Pendant l'examen, sans IA, je suis de nouveau bloquée. J'échoue encore. »

# Exercice 00 — Aaaaand... OPEN!

| Exercice : 00 |
| --- |
| **Aaaaand... OPEN!** |
| Répertoire : ex00/ |
| Fichiers à rendre : Makefile, main.cpp, ClapTrap.h ou ClapTrap.hpp, ClapTrap.cpp |
| Fonctions interdites : aucune |

Pour commencer, tu dois implémenter une classe. Quelle originalité !

Elle s'appellera ClapTrap et possédera les attributs private suivants, initialisés avec les valeurs indiquées :

- Name : le nom, transmis en paramètre au constructeur.
- Hit points (10) : les points de vie du ClapTrap.
- Energy points (10) : les points d'énergie.
- Attack damage (0) : les dégâts d'attaque.

Ajoute les public member functions suivantes :

```
void attack(const std::string& target);

void takeDamage(unsigned int amount);

void beRepaired(unsigned int amount);
```

Lorsque ClapTrap attaque, sa cible perd un nombre de points de vie égal à ses attack damage. Lorsqu'il se répare, il récupère amount points de vie. Chaque attaque et chaque réparation consomment 1 point d'énergie. ClapTrap ne peut rien faire s'il n'a plus de points de vie ou de points d'énergie.

Pour ces exercices d'introduction, les instances de ClapTrap ne doivent pas interagir directement entre elles. Les paramètres ne feront donc pas référence à une autre instance de ClapTrap.

Chaque member function doit afficher un message décrivant ce qui se passe. Par exemple, attack() peut afficher :

```
ClapTrap <name> attacks <target>, causing <damage> points of damage!
```

Les constructors et le destructor doivent également afficher un message afin que les évaluateurs puissent vérifier qu'ils ont été appelés.

Écris et rends tes propres tests pour vérifier que ton code fonctionne comme prévu.

# Exercice 01 — Serena, my love!

| Exercice : 01 |
| --- |
| **Serena, my love!** |
| Répertoire : ex01/ |
| Fichiers à rendre : Fichiers précédents + ScavTrap.h ou ScavTrap.hpp, ScavTrap.cpp |
| Fonctions interdites : aucune |

Tu vas créer un robot dérivé nommé ScavTrap, qui héritera de ClapTrap. Ses constructors, son destructor et sa fonction attack() devront afficher des messages différents.

Tes tests doivent montrer l'enchaînement des constructors et des destructors. Lorsqu'un ScavTrap est créé, le programme commence par construire un ClapTrap. La destruction se produit dans l'ordre inverse. Pourquoi ?

ScavTrap utilisera les attributs de ClapTrap. Adapte ClapTrap si nécessaire. Les attributs de ScavTrap doivent être initialisés ainsi :

- Name : transmis en paramètre au constructeur.
- Hit points : 100.
- Energy points : 50.
- Attack damage : 20.

ScavTrap possédera une capacité spéciale :

```
void guardGate();
```

Cette member function doit afficher un message indiquant que ScavTrap est maintenant en mode Gate keeper.

N'oublie pas d'ajouter des tests supplémentaires à ton programme.

# Exercice 02 — Repetitive work

| Exercice : 02 |
| --- |
| **Repetitive work** |
| Répertoire : ex02/ |
| Fichiers à rendre : Fichiers précédents + FragTrap.h ou FragTrap.hpp, FragTrap.cpp |
| Fonctions interdites : aucune |

Tu dois maintenant implémenter une classe FragTrap qui hérite de ClapTrap. Elle est très similaire à ScavTrap, mais ses messages de construction et de destruction doivent être différents.

Tes tests doivent montrer l'enchaînement des constructors et des destructors. Lorsqu'un FragTrap est créé, le programme commence par construire un ClapTrap. La destruction se produit dans l'ordre inverse.

Les attributs sont les suivants :

- Name : transmis en paramètre au constructeur.
- Hit points : 100.
- Energy points : 100.
- Attack damage : 30.

FragTrap possède également une capacité spéciale :

```
void highFivesGuys(void);
```

Cette member function doit afficher sur la standard output une demande positive de high-fives. Ajoute des tests supplémentaires.

# Exercice 03 — Now it’s weird!

| Exercice : 03 |
| --- |
| **Now it’s weird!** |
| Répertoire : ex03/ |
| Fichiers à rendre : Fichiers précédents + DiamondTrap.h ou DiamondTrap.hpp, DiamondTrap.cpp |
| Fonctions interdites : aucune |

Tu vas créer un monstre : un ClapTrap composé pour moitié de FragTrap et pour moitié de ScavTrap. Il s’appellera DiamondTrap et héritera à la fois de FragTrap et de ScavTrap.

La classe DiamondTrap doit posséder un attribut private nommé name. Cet attribut doit porter exactement le même nom de variable que celui de la classe de base ClapTrap, sans pour autant désigner le nom du robot.

- Si la variable de ClapTrap s’appelle name, celle de DiamondTrap doit aussi s’appeler name.
- Si la variable de ClapTrap s’appelle _name, celle de DiamondTrap doit aussi s’appeler _name.

Les attributs et member functions de DiamondTrap proviendront de ses classes parentes :

- Name : transmis en paramètre au constructeur.
- ClapTrap::name : le paramètre du constructeur, suivi du suffixe "_clap_name".
- Hit points : ceux de FragTrap.
- Energy points : ceux de ScavTrap.
- Attack damage : celui de FragTrap.
- attack() : celle de ScavTrap.

En plus des fonctions spéciales de ses deux classes parentes, DiamondTrap possédera sa propre capacité :

```
void whoAmI();
```

Cette member function doit afficher à la fois le nom de DiamondTrap et celui de son ClapTrap.

L’instance de ClapTrap appartenant à DiamondTrap doit être créée une seule fois. Il y a une astuce. Ajoute encore des tests à ton programme.

Le sujet attire ton attention sur les flags de compilation -Wshadow et -Wno-shadow.

Tu peux valider ce module sans terminer l’exercice 03.

# Chapitre VIII — Soumission et Peer-Evaluation

Soumets ton travail dans ton dépôt Git, comme d’habitude. Seul le travail présent dans ton dépôt sera évalué pendant la soutenance. Vérifie que les noms de tes répertoires et de tes fichiers sont corrects.

Pendant l’évaluation, une petite modification du projet pourra parfois t’être demandée : modifier légèrement le comportement du programme, écrire ou réécrire quelques lignes de code, ou ajouter une fonctionnalité simple.

Cette étape ne s’applique pas nécessairement à tous les projets. Cependant, tu dois être prête à la réaliser si elle est mentionnée dans les consignes d’évaluation.

Elle permet de vérifier que tu comprends réellement une partie précise du projet. La modification peut être effectuée dans l’environnement de développement de ton choix et doit pouvoir être réalisée en quelques minutes, sauf si un délai particulier est indiqué.

Par exemple, on peut te demander de modifier une fonction ou un script, changer un affichage, ou adapter une structure de données pour stocker de nouvelles informations.

Les détails de cette modification — son périmètre, sa cible, etc. — seront précisés dans les consignes d’évaluation. Ils peuvent varier d’une évaluation à l’autre, même pour un même projet.